Welcome to your next lesson: Python errors: meet the traceback.

I'm preparing your personalized lesson based on what you've learned, what comes next, and the best way to make it stick.

Step by step, you're moving forward.

-

Imagine you write this in the code editor and run it:

```python
age = "30"
print(age + 5)
```

Python will crash and show a block of red text. That block is called a traceback — it's Python's "crash report" that explains what went wrong and where ?

Think of it like your car's dashboard when something breaks: it lights up the exact problem so you know where to look, instead of just turning the car off with no clue.

Ron, before we dive deeper:

Have you seen a Python error message / traceback before in this course or elsewhere?

- Yes, I've seen them
- I'm not sure
- No, this is new

Yes, I've seen them

Nice — that helps a lot ?

Since you've seen tracebacks before, let's tighten up why they exist.

When Python hits something it can't do (like `age + 5` when `age` is a string), it has two choices:

1. Just stop with no info ❌
2. Stop and show a traceback that says:

- what went wrong
- where it went wrong

So a traceback is Python's way of saying:

> *"Here's the exact spot and the reason I crashed so you can fix it."*

Does that idea make sense: traceback = Python's crash report to help you debug?

Reply with "yes" or "not yet" and we'll go step by step.

Yes

Here's a visual of a simple Python traceback ?

You can hover over each part to see what it means (file name, line number, code line, error type, message).

Now I want to zoom in on just one idea:

At the very bottom line of a traceback, Python shows two things:

- the error type (like NameError, TypeError, ValueError)
- the error message (a short sentence with extra detail)

Together, they answer:

> *"What kind of problem is this, and what extra info do I need?"*

Make sense so far: bottom line = error type + message?

Reply "yes" or "not really" and we'll adjust pace.

Basic Python Traceback Layout



Yes

Perfect ✅

Now let's zoom in on one more key piece:

Some tracebacks are one line of code, some are many lines long.

But no matter how long it is:

- The bottom line always tells you the kind of error and a short message.
- One of the lines above (often just one, for now) points to the exact line of your code that crashed.

If you can find:

1. the file + line number
2. the bottom error type + message

…you can usually fix the bug much faster ?

Want to try a quick check where you just identify which part of a traceback gives the error type?

Anatomy of a Simple Python Traceback



Sure, let's do it

What do you think? Pick the option that shows the error type ?

Look at this fake traceback:

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(age + 5)
TypeError: can only concatenate str (not "int") to str
```

Which part tells you the error type?

A.

Traceback (most recent call last):

B.

File "main.py", line 2, in <module>

C.

print(age + 5)

D.

TypeError: can only concatenate str (not "int") to str

Nice job, Ron ? You nailed it.

TypeError: can only concatenate str (not "int") to str is correct because:

- TypeError is the error type → it tells you what kind of problem it is (a type mismatch).
- The rest (can only concatenate str...) is the error message → extra detail about why it failed.

So that bottom line is your “what went wrong, in words” line ?

Next step, I want to show you how to spot the file and line number in a traceback.

Ready to zoom in on that piece? (yes / not yet)

Yes, I'm ready

Awesome ?

Let's look at where in the traceback Python tells you which line of your code broke.

In this example:

Text

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(age + 5)
TypeError: can only concatenate str (not "int") to str
```

The key line for location is:

Text

```
File "main.py", line 2, in <module>
```

Step by step, that line tells you:

- File name → "main.py"
- Line number → line 2

So if you open main.py and go to line 2, you'll find the bad line (print(age + 5)).

Does this mapping make sense?

File "main.py", line 2 → "Look in file main.py, at line 2" ✓ / ✗?

Yes, that makes sense

Great ?

Let's lock this in with a quick check about the location info in a traceback.

Remember our example:

Text

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(age + 5)
TypeError: can only concatenate str (not "int") to str
```

I'll ask you one multiple-choice question: you just pick which part tells you exactly where (file + line) to look.

What do you think? ?

Sounds good

What do you think? Take your time and pick the option that shows the file + line info ?

In this traceback:

```
Traceback (most recent call last):
```

```
  File "script.py", line 4, in <module>
    print(age + 5)
TypeError: can only concatenate str (not "int") to str
```

Which part tells you exactly where in your code (file + line) the error happened?

A.

Traceback (most recent call last):

B.

File "script.py", line 4, in <module>

C.

print(age + 5)

D.

TypeError: can only concatenate str (not "int") to str

Boom, nailed it ?

File "script.py", line 4, in <module> is exactly the part that tells you where to look:

- "script.py" → which file
- line 4 → which line in that file

So your debugging "workflow" from a traceback is basically:

1. Find the file + line line ➜ open that spot in your code
2. Look at the bottom line ➜ error type + message

Those two together are your main debugging compass ?

Next, I'd like to show you a different error type (like NameError) so you can see how tracebacks all follow the same pattern.

Want to try a quick NameError example next?

Yes, let's try it

Great ? Let's keep it simple and visual.

Imagine this code in the editor:

Python

```
age = 30
print(agge)
```

Here's what happens step by step:

1. You defined age, not agge.
2. Python reaches print(agge) and says: "I don't know any variable called agge."
3. It crashes and shows a traceback whose bottom line looks like:

Text

```
NameError: name 'agge' is not defined
```

This is a NameError: Python is telling you "this name doesn't exist."

Does that idea land clearly?

NameError = you tried to use a variable (or name) that Python doesn't know about.

Yes / a bit fuzzy?

Yes, that's clear

Nice ? You've now seen two common error types:

- TypeError → mixing incompatible types (like string + int)
- NameError → using a name Python doesn't know

Even though the reason is different, the traceback shape is the same:

1. A line with File + line number → where to look
2. The code line that crashed
3. The bottom line with error type + message

I want to check that you can read a full traceback on your own now, step by step.

Next up: I'll show you a slightly longer traceback, and you'll tell me:

- which part is the file + line
- which part is the error type + message

Ready for that quick challenge? (yes / not yet)

Yes, I'm ready

Awesome, let's do this ?

Here's a slightly longer traceback:

Text

```
Traceback (most recent call last):
  File "app.py", line 3, in <module>
    print("Age: " + 7)
TypeError: can only concatenate str (not "int") to str
```

Focus on the bottom line again:

Text

```
TypeError: can only concatenate str (not "int") to str
```

Step by step:

- TypeError → the error type (wrong types used together)
- The rest → the message explaining what exactly is wrong

So even in a bigger traceback, you still read the bottom line the same way:

error type first, then the explanation.

Next, I'll give you a multiple-choice question about this traceback so you can practice spotting either the location or the error type. Ready for that quick check?

Yes, I'm ready

What do you think? Pick the line that shows the error type and its message ?

Look again at this traceback:

```
Traceback (most recent call last):
```

```
  File "app.py", line 3, in <module>
    print("Age: " + 7)
TypeError: can only concatenate str (not "int") to str
```

Which line tells you the kind of error (the error type) and its message?

A.

Traceback (most recent call last):

B.

File "app.py", line 3, in <module>

C.

print("Age: " + 7)

D.

TypeError: can only concatenate str (not "int") to str

You're on a roll, Ron ?

TypeError: can only concatenate str (not "int") to str is exactly the right line because:

- TypeError = the kind of error (type problem)
- can only concatenate str (not "int") to str = the message explaining what went wrong

So any time you see a traceback, you can jump straight to the bottom line to know:

"What kind of error is this, and what extra detail is Python giving me?"

Before we move on:

If I show you any traceback now, do you feel you can reliably find these two things?

1. File + line number
2. Error type + message

Answer honestly: "yes" or "not fully yet".

Yes, I feel confident

Love it ? That confidence is earned.

Let's lock in everything you've mastered about tracebacks so far:

- You know what a traceback is: Python's crash report, not just "scary red text".
- You can find where the error happened: the line starting with File "something.py", line X.
- You can read what kind of error it is: the bottom line (like NameError, TypeError) plus its message.

That covers the core goals for this lesson ✅

Two paths from here:

- If you'd like, I can give you one more mixed challenge traceback to read and explain.
- Or, if you feel solid and want to keep moving, we can wrap this lesson and you can go to the next one.

What do you prefer:

"one more practice" or "let's move on to the next lesson"?

We can move on. This is truly something I've seen in programming debuggers for 20-30 years so I'm pretty familiar with

That makes total sense — your experience shows ?

Quick recap of what you just locked in for Python tracebacks:

- A traceback is just Python's crash report, not an enemy.
- You know how to grab:
- Where it failed → File "...", line N
- What kind of error → bottom line (TypeError, NameError, etc.) + message
- You've connected this to what you've already seen for decades in other debuggers, so this is more of a translation, not brand-new ground.

You're all set on this lesson ✅

You can jump to the next lesson from the Learn tab whenever you're ready.

If you want, you can also paste any real traceback you hit later, and I'll walk through it with you step by step.

Ron, this lesson is a challenge that pulls together your core Python basics: arithmetic, types, input/print, formatting, and reading a simple error.

We'll do this as 6 short rounds. Each round is one focused question. I won't say right/wrong until all 6 are done — then I'll give you a quick strengths + next steps summary ?

Let's start with Round 1 (arithmetic):

A café bill is $8.50 and you leave a $2.00 tip.

In Python, you write:

```python
total = 8.50 + 2.00
print(total)
```

What number do you expect this to print?

Just type your answer as a number (for example: 10.5).